Федеральное государственное бюджетное образовательное учреждение высшего образования

«Московский государственный технический университет имени Н.Э. Баумана»
(национальный исследовательский университет)
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ

КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

О т ч е т

по лабораторной работе №1

Название лабораторной работы: Программирование с использованием библиотеки QT

Дисциплина: Алгоритмизация и программирование

Студент гр. **ИУ6-22Б** _____ **М. Д. Акимов**
(Подпись, дата) (И.О. Фамилия)

Преподаватель _____ **О. А. Веселовская**
(Подпись, дата) (И.О. Фамилия)

Москва, 2026

Цели: изучить основные принципы работы с библиотекой Qt, научиться создавать простые приложения в Qt и Qt Designer

```cpp
#include <QApplication>
#include <QWidget>
#include <QHBoxLayout>
#include <QSpinBox>
#include <QSlider>
#include <QPushButton>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QWidget *window = new QWidget();
    window->setWindowTitle("Введите Ваш Возраст");

    QSpinBox *spinBox = new QSpinBox(window);
    QSlider *slider = new QSlider(Qt::Horizontal, window);
    QPushButton *btn = new QPushButton("Завершение", window);

    spinBox->setRange(0, 130);
    slider->setRange(0, 130);
    spinBox->setValue(35);

    QHBoxLayout *layout = new QHBoxLayout(window);
    layout->setContentsMargins(5, 5, 5, 5);
    layout->setSpacing(5);

    layout->addWidget(spinBox);
    layout->addWidget(slider);
    layout->addWidget(btn);

    // Qt6 синтаксис сигналов
    QObject::connect(spinBox, &QSpinBox::valueChanged, slider, &QSlider::setValue);
    QObject::connect(slider, &QSlider::valueChanged, spinBox, &QSpinBox::setValue);
    QObject::connect(btn, &QPushButton::clicked, window, &QWidget::close);

    window->show();
    return app.exec();
}

```

Рисунок 1 - код программы

1) Напишем программу:

Результат работы программы (Схема выравнивания QHBoxLayout):

```cpp
#include <QApplication>
#include <QWidget>
#include <QVBoxLayout>
#include <QSpinBox>
#include <QSlider>
#include <QPushButton>

int main(int argc, char *argv[])
{
    QApplication app(argc, argv);

    QWidget *window = new QWidget();
    window->setWindowTitle("Введите Ваш Возраст");

    QSpinBox *spinBox = new QSpinBox(window);
    QSlider *slider = new QSlider(Qt::Horizontal, window);
    QPushButton *btn = new QPushButton("Завершение", window);

    spinBox->setRange(0, 130);
    slider->setRange(0, 130);
    spinBox->setValue(35);

    QVBoxLayout *layout = new QVBoxLayout(window);
    layout->setContentsMargins(5, 5, 5, 5);
    layout->setSpacing(5);

    layout->addWidget(spinBox);
    layout->addWidget(slider);
    layout->addWidget(btn);

    // Qt6 синтаксис сигналов
    QObject::connect(spinBox, &QSpinBox::valueChanged, slider, &QSlider::setValue);
    QObject::connect(slider, &QSlider::valueChanged, spinBox, &QSpinBox::setValue);
    QObject::connect(btn, &QPushButton::clicked, window, &QWidget::close);

    window->show();
    return app.exec();
}
```

Рисунок 3 - Программа с другим выравниванием

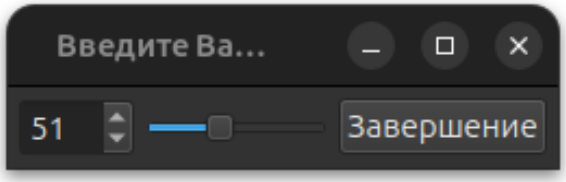


Рисунок 2 - Результат работы прогаммы

Перепишем программу, поменяв схему выравнивания на QVBoxLayout:

Результат программы стал следующим:

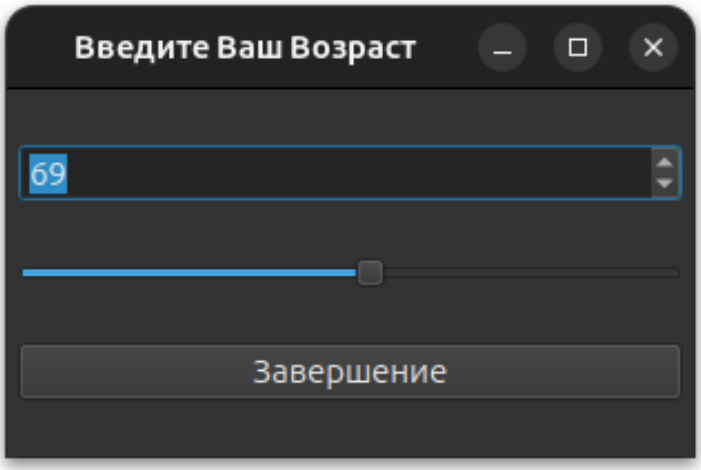


Рисунок 4: результат работы измененной программы

2) Откроем программу QtDesigner и создадим диалог без кнопок. Разместим на форме элементы согласно заданию:

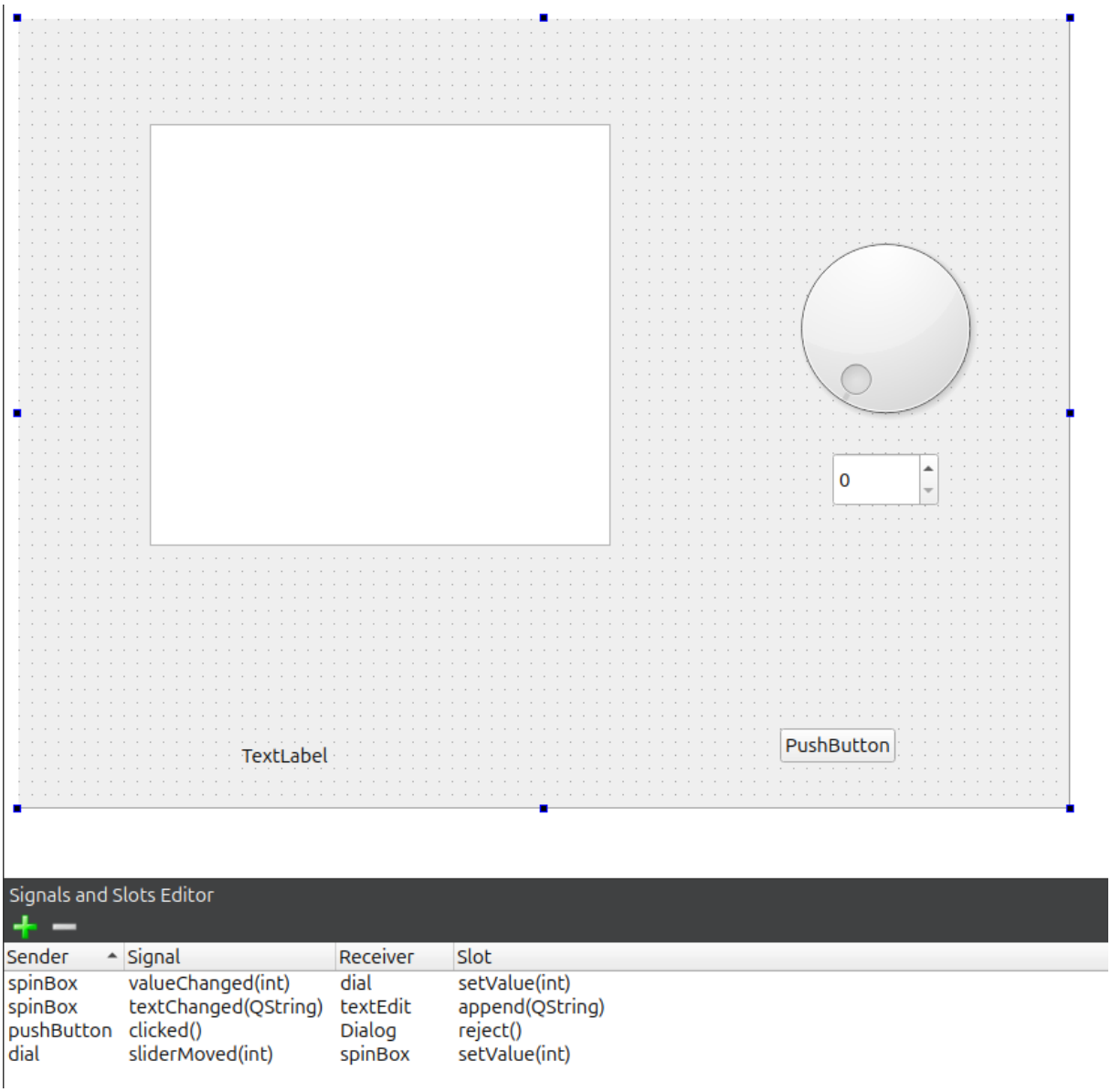


Рисунок 5 - Расположение элементов

Перейдем в режим сигналов и слотов и установим следующие соответствия:

dial, SIGNAL(**sliderMoved**(int)) – spinBox, SLOT(**setValue**(int)) spinBox, SIGNAL(**valueChanged**(int)) – dial, SLOT(**setValue**(int)) btnExit,SIGNAL(**clicked**()) – DialogEx2, SLOT(**reject**()) spinBox, SIGNAL(**valueChanged**(QString)) – textEdit, SLOT(**append**(QString))

Далее создадим заголовочный файл dialogex2.h:

```
#pragma once

#include <QDialog>
#include "ui_DialogEx2.h"

class DialogEx2 : public QDialog, public Ui::DialogEx2
{
    Q_OBJECT
public:
    DialogEx2(QWidget* parent = 0);
private slots:
    void onExitClicked();
};
```

Рисунок 6 - содержимое заголовочного файла

Опишем класс DialogEx2 в файле dialodex2.cpp:

```
#include "dialogex2.h"
#include <QMessageBox>


DialogEx2::DialogEx2(QWidget * parent) :QDialog(parent)
{
    setupUi(this);
    connect(pushButton, SIGNAL(clicked()), this, SLOT(onExitClicked()));
}

void DialogEx2::onExitClicked()
{

    if (QMessageBox::question(this, QString(), "Exit?", QMessageBox::Yes | QMessageBox::No) == QMessageBox::Yes)
    {
        exit(0);
    }
    else {
        show();
    }
}
```

Рисунок 7 - Описание класса

В отдельном файле ex2.cpp реализуем создание объекта приложения и отображение формы:

```
#include "dialogex2.h"
#include <QApplication>
#include <QSplitter>

int main(int argc, char* argv[])
{
    QApplication app(argc, argv);

    //Для создания одной формы
    DialogEx2* dialog = new DialogEx2();
    dialog->show();

    //Для создания двух форм
    QSplitter* splitter = new QSplitter(Qt::Horizontal);
    DialogEx2* dialog1 = new DialogEx2();
    DialogEx2* dialog2 = new DialogEx2();
    splitter->addWidget(dialog1);
    splitter->addWidget(dialog2);
    splitter->show();

    return app.exec();
}
```

Рисунок 8 - содержание ex2.cpp

Запускаем программу с вариантом кода для отображения двух форм:



Рисунок 9 - результат работы программы

Изменим тип разделителя с *QSplitter(Qt::Vertical)* на *Qsplitter(Qt::Horizontal)* и зафиксируем изменения:

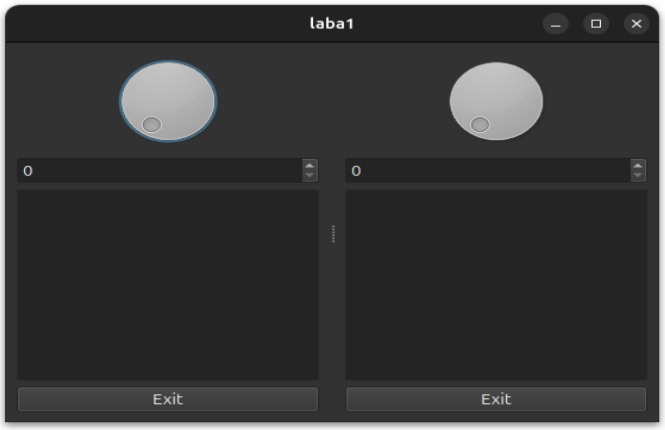


Рисунок 10 - результат работы изменённой программы

После изменения типа разделителя формы стали располагаться по очереди горизонтально, а не вертикально:

3)
Напишем реализацию класса CalcDialog в заголовочном файле calcDialog.h:

```
#ifndef CALCDIALOD_H
#define CALCDIALOD_H

#include <QDialog>
#include <QLineEdit>
#include <QSignalMapper>

class CalcDialog : public QDialo
{
    Q_OBJECT
public:
    CalcDialog(QWidget* parent =
    virtual ~CalcDialog() {};
protected:
    QSignalMapper* m_pSignalMapp
    QLineEdit* m_pLineEdit;
    double m_Val;
    int m_Op;
    bool m_bPerf;
    void initNum();
    double getNumEdit();
    void setNumEdit(double);

    void calcPrevOp(int curOp);


    void checkOpPerf();
private slots:
    void clicked(int id);
};

#endif // CALCDIALOD_H
```

Результат работы программы:

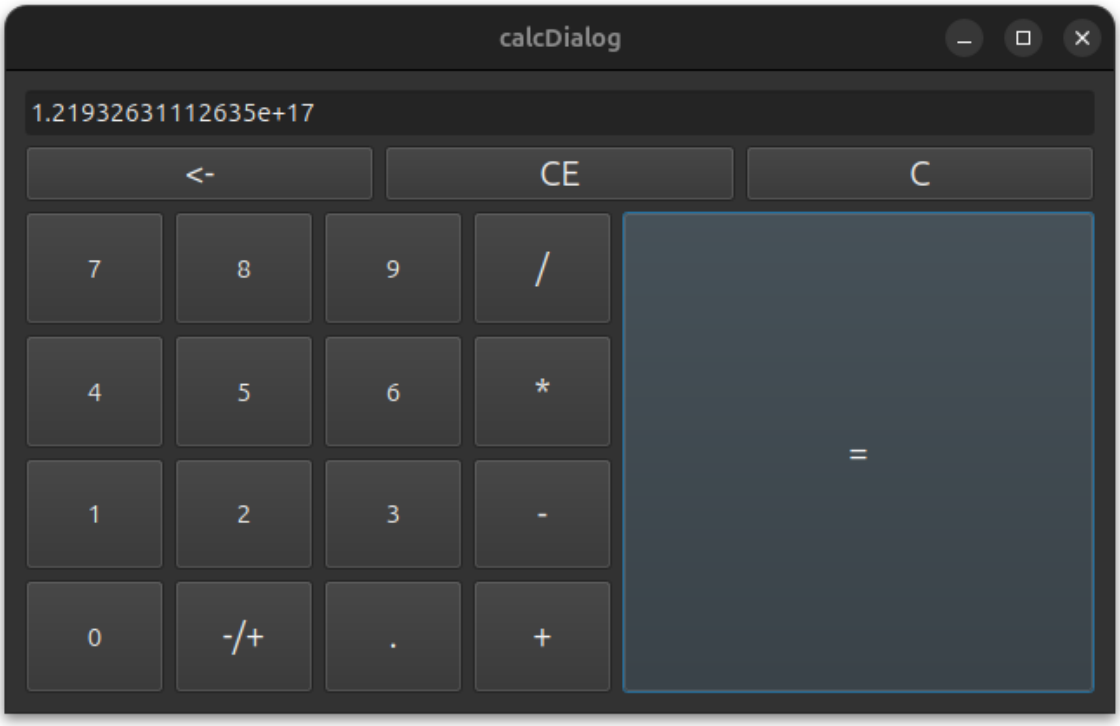


Рисунок 12 - Результат работы программы

Чтобы добавить четыре новые кнопки слева от цифровых кнопок вертикально, выполним следующие шаги:

1) Добавим 4 идентификатора соответствующих кнопок, далее в массив _btnDescr добавим 4 соответствующих элемента, создадим 2 новые схемы выравнивания (для колонки новых кнопок и горизонтальной связи этой колонки с панелью цифровых кнопок):

```
#define STEP -1
#define LOG -2
#define SIN -3
#define COS -4
_btnDescr.push_back(BtnDescr("x^y", STEP));
_btnDescr.push_back(BtnDescr("logxY", LOG));
_btnDescr.push_back(BtnDescr("sin", SIN));
_btnDescr.push_back(BtnDescr("cos", COS));

QVBoxLayout* dopLayout = new
QVBoxLayout(); QHBoxLayout* allLayout
= new QHBoxLayout();
```

4) Обеспечиваем соответствие кнопки ее идентификатору:

m_pSignalMapper->setMapping(button, _btnDescr[i].id);

```
    if( i >= GRID_KEYS + 3 || i < GRID_KEYS)
        button->setSizePolicy ( QSizePolicy::Expanding,QSizePolicy::Expanding);

    if( _btnDescr[i].id >= 10 ){
        QFont fnt = button->font();
        fnt.setPointSize( fnt.pointSize () + 4 );
        button->setFont( fnt );
    }

    connect(button, SIGNAL(clicked()), m_pSignalMapper, SLOT(map()));

    m_pSignalMapper->setMapping(button, _btnDescr[i].id);
    if(i<GRID_KEYS)
        gridLayout->addWidget(button, i / 4, i % 4);
    else if( i < GRID_KEYS + 3)
        bccKeysLayout->addWidget(button);
    else if ( i == 19)
    {
        mainKeysLayout->addLayout(gridLayout);
        mainKeysLayout->addWidget(button);
    } else {
        newLayout->addWidget(button);
    }
}
```

Рисунок 13 - соединение кнопки с действием

5)Добавляем блоки кнопок в схему выравнивания всей формы:

allLayout->addLayout(dopLayout); allLayout->addLayout(mainKeysLayout); dlgLayout->addWidget(m_pLineEdit); dlgLayout->addLayout(bccKeysLayout); dlgLayout→addLayout(allLayout);

6) Добавляем команды для обработки нажатия новых клавиш:

```
case SIN:
{
  setNumEdit(sin(getNu
  mEdit())); break;
}
case COS:
{
  setNumEdit(cos(getNu
  mEdit())); break;
}
case LOG:
case STEP:
```

7) Добавляем команды для обработки бинарных операций:

```
case LOG: {
  m_Val = log(m_Val) /
  log(num); break;
}
```

```
case STEP: {
  m_Val = pow(m_Val,
  num); break;
}
```
Результат работы измененной программы:

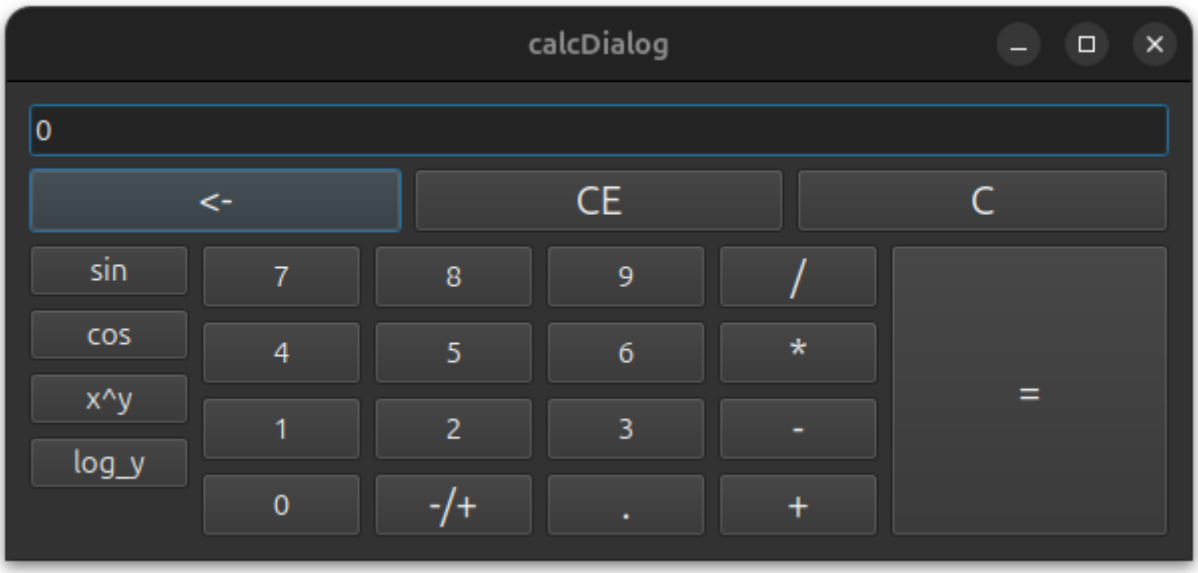


Рисунок 14 - Результат работы измененной программы

Слева от цифровых кнопок добавились кнопки четырех новых операций.

Напишем реализацию класса MyWidget в заголовочном файле

mywidget.h:

```
#include <QWidget>

#include <QLineEdit>

#include <QTextEdit>

#include <QPushButton>

#include <QVBoxLayout>


class MyWidget : public QWidget {

   Q_OBJECT
```

```
public:
    MyWidget(QWidget *parent = 0);
private slots:
    void onEditingFinished();
    void onBtnClicked();
private:
    QLineEdit *m_pEdit;
    QTextEdit *m_pText;
    QPushButton *m_pBtn;
    bool m_bUpper;
};
```

Реализуем методы класса в файле mywidget.cpp:

```
#include "mywidget.h"

MyWidget::MyWidget(QWidget *parent) : QWidget(parent), m_bUpper(true)
{
    m_pEdit = new QLineEdit(this);
    m_pText = new QTextEdit(this);
    m_pBtn = new QPushButton(tr("Convert"), this);
    m_pText->setReadOnly(true);

    QVBoxLayout *layout = new QVBoxLayout(this);
```

```
    layout->addWidget(m_pEdit);
    layout->addWidget(m_pBtn);
    layout->addWidget(m_pText);

    connect(m_pEdit, SIGNAL(editingFinished()), this,
SLOT(onEditingFinished()));
    connect(m_pBtn, SIGNAL(clicked()), this, SLOT(onBtnClicked()));
}

void MyWidget::onEditingFinished() {
    QString str = m_pEdit->text();
    m_pText->append(tr("input: ") + str);
    m_pEdit->clear();
}

void MyWidget::onBtnClicked() {
    QString str = m_pText->toPlainText();
    if(m_bUpper) m_pText->append(str.toUpper());
    else m_pText->append(str.toLower());
    m_bUpper = !m_bUpper;
}
```

3. В файле main.cpp реализуем отображение формы:

cpp

```
#include <QApplication>
#include "mywidget.h"

int main(int argc, char *argv[]) {
    QApplication a(argc, argv);
    MyWidget w;
    w.show();
    return a.exec();
}
```

Результат работы программы:

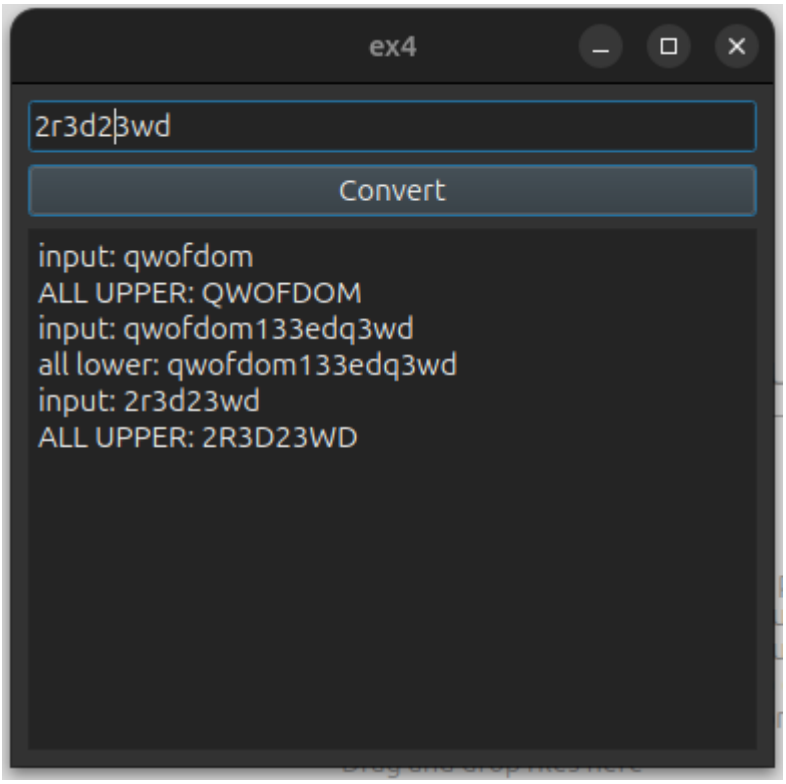


Рисунок 15 - Результат работы программы

Вывод: В ходе выполнения лабораторной работы были освоены принципы работы с элементами ввода-вывода QLineEdit и QTextEdit, а также закреплены навыки программного создания интерфейса и управления компоновкой с помощью QVBoxLayout без использования Qt

Designer. Практическая реализация обработки сигналов editingFinished и clicked позволила изучить механизмы взаимодействия пользователя с программой, а также закрепить навыки управления состоянием виджетов и динамической обработки текстовых данных в многоязычных приложениях Qt.